

Research Progress of Machine Learning in Software Defect Prediction

Fanqi Meng

School of Computer Science
Northeast Electric Power University
Jilin China
mengfanqi@neepu.edu.cn

Yongjun Li*

School of Computer Science
Northeast Electric Power University
Jilin China
2031600130@qq.com

Jingdong Wang*

School of Computer Science
Northeast Electric Power University
Jilin China
707569380@qq.com

Abstract—Software defect prediction is of great significance to ensuring software security. Appropriate metrics are an effective way to predict whether a program contains software flaws. In recent years, machine learning techniques have been widely used to measure programs to predict software defects and have achieved good results. In order to clarify the research progress of software defect prediction based on machine learning, this paper reviews the research results in this field in the past three years from two aspects: within-project defect prediction and cross-project defect prediction. Firstly, the software defect prediction framework based on machine learning and its development are introduced. Secondly, this paper summarizes the research results in the field of defect prediction in the past three years from two aspects: within-project defect prediction and cross-project defect prediction. Finally, based on the research results so far, the status and hot spots of machine learning in software defect prediction are discussed, which provides possible help for the development direction of machine learning in software defect prediction.

Keywords—component; machine learning; defect analysis; software defect prediction; cross project defect prediction; software security

I. INTRODUCTION

Software defects are problems, errors, or hidden functional defects in computer software or programs that destroy the ability of the software or program to run normally. Software defect prediction can provide risk warnings for software modules with defects, which is essential to ensure software security. In the early stage of software defect prediction, some researchers used lines of code as a metric for software defect prediction to obtain a rough prediction effect. With the rapid rise of the software industry, the number of lines of software code is also increasing rapidly, and the effect of software defect prediction models based on traditional methods is poor. Machine learning is a learning method that relies on a large amount of data to train and automatically improve. The more data, the better the effect of model training. With the introduction of machine learning technology into software defect prediction, researchers have developed many prediction models with excellent performance. The research on software defect prediction has achieved many results.

Since the release of PROMISE, the first software defect data warehouse, in 2005, machine learning has been further developed in software defect prediction [1]. Liu Yang [2] summarized the achievements of machine education from

1992 to 2004 and considered introducing new advances in machine learning, such as statistical learning theory, into the field of software defect prediction. Chen Xiang et al. [3] summarized and sorted out the static defect prediction technology and deeply summarized the defect prediction based on code modification. The development of software defect prediction technology can be divided into static defect prediction technology and dynamic defect prediction technology. Software static defect prediction technology can be divided into three directions: defect trend prediction, defect quantity prediction, and defect severity prediction. In recent years, represented by machine learning of artificial intelligence technology significantly promoted the development of software defect prediction research. Based on the existing research, this paper profoundly summarizes the process of machine learning defect prediction, intra-project defect prediction, and cross-project defect prediction. It discusses the possible research directions of machine learning in software defect prediction.

II. LITERATURE SELECTION AND RELATED WORK

A. Literature Selection

This paper combs through a large amount of literature related to machine learning in software defect prediction, focusing on two directions of software defect prediction: within-project defect prediction and cross-project defect prediction. These directions have emerged a large number of machine learning research results, which can effectively show the progress of machine learning technology.

Based on the above steps, a large amount of literature was selected from the scientific platform of China Knowledge Network and the Web of Science website. There are 32,500, 35,100, and 34,500 papers on machine learning in 2020, 2021, and 2022, respectively. Machine learning is widely used in many fields and has much-related literature. Machine learning techniques continue to be hot in the research of software defect prediction. There is still great potential for research on software defect prediction based on machine learning.

B. Related Work

At present, the rapid growth of software scale has put forward higher requirements for software defect prediction technology, especially the accuracy of defect prediction models. Artificial intelligence technology, represented by machine learning, has also achieved many excellent

prediction models in software defect prediction. This paper introduces some important machine learning models, analyzes their pros and cons, and explains the basics of software defect prediction. The development of machine learning technology in the field of software defect prediction is summarized, and the problems of software defect prediction technology based on machine learning are presented. Finally, the problems and possible development directions of software defect prediction technology based on machine learning are discussed.

III. RESEARCH METHODS

A. Software Defect Prediction Architecture Based on Machine Learning

The software defect prediction framework based on machine learning can be divided into three parts: data extraction, model training and model application.

It is necessary to collect code data in the software development process and select the appropriate metrics for the data extraction stage. This stage is to collect and process the defective data in the software. Typically, the defect module code in the historical version of the software is extracted and marked to form the training model data set.

In the model training phase, the data set obtained in the data extraction phase is divided into a training set and a test set. Appropriate machine learning techniques (such as Naive Bayes, decision tree, etc.) are selected to establish a preliminary defect prediction model on the training set. After the preliminary prediction model is obtained, the model is applied to the test set to obtain the prediction results, and the prediction effect of the model on the test set is evaluated, so that the parameter optimization model is further adjusted. Accuracy and time spent can be used to evaluate the effect of the model.

In the model application stage, new software data is made and processed for use in the optimized model for prediction, and the prediction target may be the number of defects or defect tendency or defect severity.

In defect prediction, historical defect data must be distinguished. In practice, a standard method is to mark the data (usually source code) as defective or non-defective and then select the appropriate machine learning algorithm according to the characteristics of the data. The early defect prediction model is trained using data from historical versions of the project and is, therefore, also known as in-project defect prediction (WPDP).

In software development, historical software versions need more data to train high-performance defect prediction models. Some researchers have proposed cross-project defect prediction (CPDP) to solve this problem. CPDP uses the defect data of other projects to train the prediction model, solves the problem that the software defect prediction model relies heavily on historical data, realizes the reuse of data, and has a milestone significance in the development of software defect prediction technology. At the same time, CPDP has problems with data distribution differences, noisy data, and data privacy. Some researchers have proposed the concept of transfer learning, aiming at the poor effect of CPDP directly using data from other projects. Transfer learning is about

finding the data that contributes cross projects and reusing it. In addition, some researchers have proposed federated learning to protect data privacy. Federated learning is a distributed machine learning technique that allows collaboration without exchanging data. In federated learning, there is a big model aggregated by the data of all parties. Then each collaborator applies it according to their needs, avoiding disputes over sensitive data.

In order to ensure the effectiveness of model training, data are often divided into training sets and data sets. Establishing a complete model requires evaluating the model effect and improving it many times.

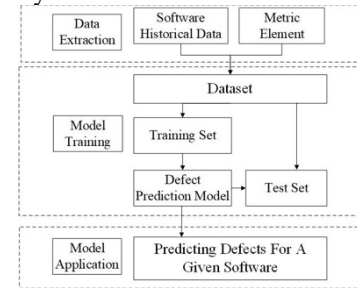


Figure 1. Model Framework of Software Defect Prediction based on Machine Learning

B. Machine Learning Technology in Defect Prediction

Machine learning plays a vital role in software defect prediction. According to the different learning methods, machine learning can be divided into supervised learning and unsupervised learning. Supervised learning requires training data with labels, finding defective features in the training data to build a model. The classification algorithm is one of the widely used supervised learning algorithms. Unsupervised learning deals with unlabeled data, i.e., finding hidden patterns or components from the data set during the model training phase. The clustering algorithm is a standard unsupervised algorithm.

Depending on the data source from which the model is trained, software defect prediction can be divided into WPDP and CPDP. According to the model's training mode, it can be divided into supervised, unsupervised, and semi-supervised learning.

In defect prediction, it is imperative to adopt the appropriate metric when extracting the characteristics of the training data to build the model. Metrics refer to intrinsic attributes related to the external performance of defects, and these attributes are software metrics data [4]. Software metrics are the quantification of the properties of software code. In addition, with the diversified development of computer programming, a wide variety of programming languages have been born, and the logic of software code developed based on these programming languages is different. Therefore, in order to ensure the accuracy and efficiency of the defect prediction model, it is often necessary to adjust the metric attributes according to the different software development platforms.

IV. MACHINE LEARNING FOR WITHIN-PROJECT DEFECT PREDICTION

The training data used in the WPDP method is the data information of the software module in the historical version of the project. The training data and test data belong to the same software project, and the sample distribution of the training data and test data is the same [5]. The WPDP model has relatively low training difficulty and good results. At present, the development of WPDP technology is more mature. Many classical machine learning algorithms perform well on WPDP, such as a k-nearest neighbor, decision tree, random forest, and Naive Bayes. This section will briefly introduce their development achievements in turn and summarize the current research results of WPDP.

A. K-nearest Neighbors Algorithm

The k-nearest neighbor algorithm is a relatively simple machine-learning classification algorithm. The training process could be more transparent. The main idea is to give a labeled training set of data, input to classify the data, and then compare the characteristics of the data similar to the top k data in the training set. Determine the category where these k data occur most, and then the data has the same classification as these k data. The Euclidean distance or Hamiltonian distance is a standard metric used to measure similarity in k-nearest neighbors, which is calculated as follows:
Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (1)$$

Manhattan distance:

$$d(x, y) = \sqrt{\sum_{i=1}^n |x_i - y_i|} \quad (2)$$

In recent years, Chang Ruihua [6] et al. introduced the idea of density into the nearest neighbor algorithm. They proposed a software defect prediction method based on affinity propagation with hyper-Euclidean distance, which improved the prediction accuracy of unlabeled software defect data.

B. Decision Tree Algorithm

The decision tree algorithm is a classification algorithm using a tree structure containing root, inner, and leaf nodes. The root node contains all the data, the internal nodes are usually some judgment rules, and the leaf nodes are the final classification results. This is a logical classification algorithm, through each layer of data judgment to classify. Commonly used decision tree algorithms include ID3, C4.5, and CART algorithms. The ID3 algorithm selects the features with the highest information gain for classification, while C4.5 uses the information gain rate compared to ID3, which overcomes ID3's disadvantage of emphasizing the number of features. The CART algorithm further simplifies the operation process and judges the impurity of the model by the Gini coefficient. Numerous algorithms derived from decision

tree models have been developed and extensively applied for software defect prediction.

C. Random Forest Algorithm

The random forest algorithm is made up of many decision trees, but these decision trees are not related to each other. The data is imported into the random forest algorithm, where each decision tree produces a classification result. Finally, the results of all decision trees in the forest are counted for classification. This algorithm is less prone to overfitting and balancing errors in the data. In 2018, Xue Canguan [7] et al. proposed an improved forest algorithm to improve prediction performance and efficiency. Random forest algorithms can determine the importance of features, but the classification effect is poor when the attribute value is more desirable.

D. Naive Bayes Algorithm

The Naive Bayes algorithm is based on the Bayes principle, which assumes that objects are independent of each other. Then the posterior probability can be used to determine the class. If X and Y are independent, then:

$$P(X|Y)P(Y) = P(Y|X)P(X) \quad (3)$$

Therefore, in defect prediction, if X is an unknown sample and Y is a particular class, it can be calculated whether X is defective by the above formula.

At present, the development of WPDP technology has been very mature. Iqra Batool[8] et al. conducted a systematic review of the application of machine learning technology to software defects in the review literature. They pointed out that a decision tree is a commonly used algorithm in machine learning.

Sushant Kumar Pandey[9] et al. reviewed 154 software defect prediction articles from 1990 to June 2019. According to their data report, the learning techniques mainly used for these categories of software defect prediction are random forest, support vector machine, naive Bayes, C4.5, and C4.5. Logistic Regression and Multilayer Perceptron. They also point out that the naive Bayes technique is a standard machine learning model for software defect prediction with effective results. In addition, their report considers decision tree-based models more robust and more straightforward. More suitable for small projects and unstable large projects.

At present, the development of WPDP technology is relatively mature. However, the overall performance of the defect prediction model could be better, and the research on defect severity in defect prediction could be a lot higher. Machine learning-based defect prediction models require a large amount of manually labeled data. When the iterative update of the software is significant, the prediction effect of the defect model becomes poor due to the significant difference between the historical data and the new version.

V. MACHINE LEARNING FOR CROSS-PROJECT DEFECT PREDICTION

Although the model trained by the supervised learning algorithm performs well, it requires a large amount of data from the same historical version of the software as the support. However, the software developed by most companies does not have enough historical data to train the model. To solve this problem, some researchers consider using labeled defect data from other companies to train the model and then apply the model to specific modules in their software to solve the problem of lack of training data. This method of training predictive models using data from other projects is called the CPDP technique.

At present, this method of defect prediction is popular with researchers, and there are many free and public datasets on the Internet, such as NASA datasets. Because the CPDP needs to label data manually, it consumes more time and energy in the case of large amounts of data. Researchers have proposed the concept of transfer learning to solve this problem. Unlike machine learning techniques, which require training and testing data distributions, transfer learning seeks to build accurate predictive models under different data distributions. In addition, transfer learning can achieve secondary utilization of labeled data and reduce workload. Therefore, it is widely used in CPDP, and the effect is remarkable.

Fan Guisheng [10] et al. collected 15 open-source Java projects as experimental datasets. They noted that the accuracy of instance annotation could significantly impact subsequent instance migration of source projects and the construction of defect models. In addition, they also noted that model performance would be affected if source items that may generate noisy data were not denoised. Therefore, they proposed a two-stage CPDP method to filter noisy data in the instance filtering stage and redistribute weights to reduce the difference in data in the instance migration stage, respectively. Experimental results show that this method improves the performance of software defect prediction, but the universality of this method still needs to be verified.

Gai Jinjing [11] et al. proposed a cross-project software defect prediction method based on Jensen-Shannon divergence and relative density. It believes that the accuracy of current methods based on similarity selection needs to be improved, and training without data processing will lead to a significant false positive rate. They selected the open-source projects in the PROMISE library to verify the results as the audit data set and tested the effect of eight standard classifiers. The results show that this method can improve the defect prediction model's performance and high adaptability to different classifiers.

Ni Chao [12] et al. proposed a CPDP method based on feature and instance transfer. Secondly, with the help of a small number of labeled instances in the target project, instances with similar distributions to these instances are selected from the source project. Finally, the model is established and verified on the Relink and AEEEM datasets, and the experimental results prove the model's effectiveness.

In 2021, F. Meng [13] et al. proposed a semi-supervised software defect prediction model based on Tri-training to

simultaneously address the problems of insufficient samples and class imbalance. The feature normalization method is used to smooth the data, and the SOMTE sampling method alleviates the imbalanced classification of labelled samples. In addition, they conducted experiments on four NASA public datasets, and the experimental results proved the model's effectiveness and outperformed Decision Tree, Naive Bayes, AdaBoost, and S4VM+ algorithms.

Quanyi Zou [14] et al. proposed a novel dual marginalization denoising autoencoder joint feature representation CPDP method, which combines joint feature representation learning and advanced distribution matching. They argue that joint deep representation is more effective for CPDP than just considering global feature representation methods, and advanced distribution matching is more effective in adapting probability distributions in CPDP. In addition, they evaluated the effectiveness of their approach by conducting experiments on ten open-source projects, including 29 software versions from the PROMISE repository. Experimental results show that the proposed method outperforms several state-of-the-art baseline CPDP methods.

Currently, CPDP technology has become one of the hot areas of defect prediction research, which solves the problem of the need for more data when training defect prediction models for small and medium-sized software. However, CPDP is often less effective than project defect prediction. On the one hand, the difference in data distribution between projects is challenging to eliminate. On the other hand, finding common ground between different projects could be better. In addition, cross-project data privacy security is also an urgent issue.

VI. RESEARCH HOTSPOTS

Software defect prediction technology is an essential means to ensure software quality. After years of development, many researchers have established excellent software defect prediction models and made outstanding contributions. This paper introduces the model construction framework process of machine learning in software defect prediction, introduces classical machine learning methods in software defect prediction, and summarizes the previous research results. However, some problems still need further study in software defect prediction technology based on machine learning.

A. Distributed Software Defect Prediction Model

With the rapid development of software, the value of software data is immeasurable. The same type of company is in a competitive relationship, and data often involves company secrets and cannot be leaked. There needs to be more research on defect prediction models that can protect data privacy. Distribution-based defect prediction technology can physically ensure data information security and realize cross-project data reuse. At present, there are few studies in this field, and it has great research potential.

B. More Efficient Unsupervised or Semi-supervised CPDP Models

In the current cross-project software defect prediction research, researchers have proposed some defect prediction models based on semi-supervised learning and achieved good prediction results. However, there need to be more studies, and the current CPDP effect needs to be improved. Data migrated from other projects may affect model performance. The unsupervised software defect prediction model can effectively reduce the impact of manual data processing. There needs to be more research on unsupervised or semi-supervised defect prediction technology. Unsupervised or semi-supervised defect prediction models also have great potential for mining.

C. Software Defect Prediction Model Based on Deep Learning

Deep learning is a learning method further developed based on machine learning, which significantly affects hidden mining features of data. Deep learning is excellent for fitting functions over large amounts of data. Deep learning also has mighty expressive power in feature selection, which can reduce or even avoid manual feature settings. There still needs to be more research on defect prediction based on deep learning, and the effect of mining the deep features of software source code needs to be improved.

D. Software Defect Severity Propensity Prediction Model

Currently, most of the research on software defect prediction is about defect numbers and defect tendencies. The study of bug severity can effectively help software maintenance specialists locate the bugs that most affect the regular operation of the software and should be fixed first. It can improve the efficiency of defect repair personnel in fixing defects.

VII. CONCLUSION

In this paper, we refer to the research results of machine learning in the field of software defect prediction at home and abroad in the past three years and systematically summarize them. Firstly, the framework and development of software defect prediction based on machine learning are summarized. Secondly, the research results of the past three years are reviewed in terms of within-project defect prediction and cross-project defect prediction according to the sources of software defect prediction data. Finally, the research hotspots and possible development trends of machine learning in software defect research are discussed by researchers in this field. Although machine learning has made many achievements in software defect prediction, there are still many problems to be solved. Distributed software defect prediction models, unsupervised or semi-supervised methods,

deep learning-based software defect prediction, and the severity of software defect prediction deserve further attention.

ACKNOWLEDGMENT

This work is supported by the Jilin Province Education Department project "Research on Automatic Software Defect Assignment Method Based on Multi-Feature Fusion".

REFERENCES

- [1] Yu Qiao. Research on software defect prediction method based on machine learning [D]. China University of Mining and Technology, 2017.
- [2] Liu Yang. Software defect prediction based on machine learning [J]. Computer Engineering and Application, 2006 (28): 49-53.
- [3] Chen Xiang, Gu Qing, Liu Wangshu, Liu Shulong, Ni Chao. Research on static software defect prediction [J]. Journal of Software, 2016,27 (01): 1-25. DOI: 10. 13328/J. CNKI. Jos. 004923.
- [4] Gong Lina, Jiang Shujuan, Jiang Li. Research progress of software defect prediction technology. Journal of Software, 2019, 30 (10): 3090-3114. DOI: 10. 13328/J. CNKI. Jos. 005790.
- [5] Fang Yongke, Ji Haijin, Lei Yao, Yang Shan, Xu Huiyan. Application of machine learning method in software defect prediction. Computer Knowledge and Technology, 2022, 18 (25): 47-48 + 56. DOI: 10. 14004/J. CN ki. CK t. 2022.1645.
- [6] Chang Ruihua, Shen Xiaowei. Software defect prediction method based on super Euclidean distance affinity propagation [J]. Computer Application Research, 2017, 34 (05): 1384-1387.
- [7] Xue Visited, Yan Xuefeng. Software defect prediction based on improved deep forest algorithm [J]. Computer Science, 2018, 45 (08): 160-165.
- [8] Batool I, Khan T A. Software fault prediction using data mining, machine learning and deep learning techniques: A systematic literature review[J]. Computers and Electrical Engineer, 2022,100:107886. 1065467.
- [9] Pandey S K, Mishra R B, Tripathi. Machine learning based methods for software fault prediction: A survey[J]. Expert Systems with Applications, 2021, 172:114595.
- [10] Fan Guisheng, Xi Xuyang, Yu Huiqun, Chen Liqiong. Cross-project software defect prediction method based on strength filtering and migration [J]. Computer Engineering, 2020, 46 (08): 197-202 + 209.D0I: 10.19678/J. ISSN. 1000-3428.0055054.
- [11] Gai Jinjing, Zheng Shang, Yu Hualong, Gao Shang. A method for selecting training data of CPDP source project [J]. Journal of Nanjing Normal University (Natural Science Edition), 2022, 45 (01): 110-117.
- [12] Ni Chao, Chen Xiang, Liu Wangshu, Gu Qing, Huang Qiguo, Li Na. CPDP method based on feature migration and instance migration [J]. Journal of Software, 2019, 30 (05): 1308-1329. DOI: 10. 13328/J. CNKI. Jos. 005712.
- [13] F. Meng, W. Cheng and J. Wang, "Semi-supervised Software Defect Prediction Model Based on Tri-training," KSII Transactions on Internet and Information Systems, vol. 15, no. 11, pp. 4028-4042, 2021. DOI: 10.3837/tiis.2021.11.009.
- [14] Zou Q, Lu L, Yang Z, et al. Joint feature representation learning and progressive distribution matching for cross-project defect prediction[J]. Information and Software Technology, 2021, 137: 106588.